Phage
SECURITY

SECURITY REVIEW · PREPARED FOR

Trepa

A Solana short-horizon forecasting game

DATE
07.05.2026

CONDUCTED BY
Pyro, Security Researcher
Yaneca B, Security Researcher
Zuhaibmohd, Security Researcher
Kyan, Security Researcher
Ishwar Kumar, Security Researcher

COMMIT
7724e94

REMEDIATION
6ad63ee

01 ABOUT

Phage Security

Phage Security is a team of highly skilled smart-contract security researchers collaborating with 10+ proven freelancers who have excelled in public contests and private audits alike.

With numerous vulnerabilities uncovered and patched across our completed audits, we strive to deliver the absolute best security service and client experience. While 100% security can never be guaranteed, we are committed to giving our utmost effort to protect your protocol.

PREVIOUS WORK
github.com/phage-security

REACH OUT
Telegram · @Pyro3b

WHAT AN AUDIT IS

An audit is a time, resource, and expertise-bound effort in which trained researchers evaluate smart contracts using a combination of manual and automated techniques to identify as many vulnerabilities as possible.

An audit can reveal the presence of vulnerabilities, but it cannot guarantee their absence. This report is a point-in-time assessment of the code at the commits named on the cover. It is not a warranty that the system is secure.

02 EXECUTIVE SUMMARY

FINDINGS	FIXED & VERIFIED	ACKNOWLEDGED
15	14	1

SEVERITY DISTRIBUTION

2 High	6 Medium	7 Low

CLIENT	REPOSITORY	METHODS
Trepa	github.com/TrepaOrg/trepa-onchain-svm	Manual review

CONTRACTS IN SCOPE

trepa-onchain-svm/programs/trepa/src/lib.rs	trepa-onchain-svm/programs/trepa/src/utils.rs
trepa-onchain-svm/programs/trepa/src/state.rs	trepa-onchain-svm/programs/trepa/src/context.rs
trepa-onchain-svm/utils/**	trepa-api/libs/shared/src/services/web3.service.ts

TIMELINE · APRIL 2026

Audit · 5 days		Remediation · 9 days	
KICK-OFF	END OF AUDIT	REMEDIATIONS START	REMEDIATIONS END
22.04.2026	26.04.2026	27.04.2026	05.05.2026

STARTING COMMIT HASH	7724e9400bc0fc8eee170fa008acbc76216e3ba2
FINAL COMMIT HASH	6ad63ee8f30a4a20e71dac0835680ec8042ed87d

03 PROJECT SUMMARY

What Trepa is

Trepa is a Solana short-horizon forecasting game where every player pays the same fixed entry fee and submits a numeric price estimate for an asset such as Bitcoin.

After settlement, each player's error against the realized price is compared to the field median. Players below the median win the round, get their entry back, and share a prize pool funded from losers' entries.

The on-chain program stays minimal, holding stake custody and Merkle-bound payouts, and pushes reward math, oracles, and indexing to the backend.

SECURITY POSTURE

BEFORE REMEDIATION 7724e94

Oracle trust

H-01 · M-01 · L-02

The provider oracle accepted a truncated trades page as the latest price, the indexer skipped event-order checks, and Pyth volatility used partial candles.

Settlement math drift

M-02 · M-05 · L-06

WASM waterfilling misrouted uncapped-winner dividends to takeout, a hardcoded `max_roi` could strand a pool, and `protocol_fee` was unbounded at finalization.

Claim-flow idempotency

H-02 · M-03 · M-04

Streak claims were not tied to a reward id, settlement was not idempotent under RPC failures, and non-qualifying streak users were not reset until someone else won.

On-chain robustness and signer scoping

M-06 · L-01 · L-03 · L-04 · L-05 · L-07

Token-2022 ATA derivation used the legacy SPL form, vault donations could brick finalization, account sizing used `mem::size_of`, JWT expiry was 1000x too long, and four privileged signers shared one backend process.

AFTER REMEDIATION 6ad63ee

Fourteen of fifteen findings fixed. The team shipped fixes across both repositories, and several patches went beyond the literal bug. The provider oracle now queries a single nearest-before trade with drift bounds, and streak claims bind the signed transaction and an on-chain replay PDA to the reward id, closing the double-payout primitive. Reward distribution is cap-aware, Pyth coverage is validated, and the `max_roi` and fee-bounding gaps are closed.

The one open item, L-05, is tracked as an ongoing infrastructure evaluation: the team is assessing remote signing services to separate resolver key custody from the backend. It is an operational hardening item, not a loss-of-funds issue, and the codebase is in good shape for mainnet.

04 RECOMMENDATIONS

Beyond this audit

Four measures that extend the protection of this review after launch, independent of any individual finding.

R-01	**Bug bounty**	Stand up a public bounty on Immunefi or HackenProof with the top tier covering fund loss, keeping third-party review going beyond the audit window.
R-02	**Invariant monitoring**	Run an off-chain watcher that reconciles `total_stake` against the pool ATA balance, checks `claims_left` and `open_predictions_left` consistency, and tracks `prediction_revisions` drift between chain and indexer, paging on any divergence.
R-03	**Resolver key hygiene**	Document the signing-service policy, rotation procedure, and an incident-response runbook for a compromised resolver, creator, or admin key, and keep the resolver on isolated custody.
R-04	**Lift invariants on-chain**	Continue moving critical invariants onto the on-chain program, validate oracle and indexer freshness instead of assuming it, and keep money-moving flows idempotent, so correctness does not rest entirely on the resolver-signed boundary.

05 CONTRACTS & ROLES

Core contracts

CONTRACT	ROLE
`trepa-onchain-svm/programs/trepa/src/lib.rs`	Anchor program entrypoints for the pool lifecycle (predict, update, finalize, claim, close), with per-winner ROI enforcement and Merkle-bound payouts.
`trepa-onchain-svm/programs/trepa/src/state.rs`	Account state for config, pools, predictions, and streak accumulators, holding `total_stake`, `claims_left`, and `prediction_revisions`.
`trepa-onchain-svm/programs/trepa/src/context.rs`	Anchor account contexts that gate each instruction on the correct admin, creator, or resolver signer and validate token accounts.
`trepa-onchain-svm/programs/trepa/src/utils.rs`	Shared on-chain helpers, including the associated-token-account validation used at finalization.
`trepa-onchain-svm/utils/**`	Off-chain TypeScript helpers that build finalize and claim transactions before resolver signing.
`trepa-api/libs/shared/src/services/web3.service.ts`	Backend signing service that holds the operational signers and relays finalize, claim, and streak flows to the chain.

Backend audited at b1a8ae3 and remediated at c0c3d3f.

PRIVILEGED ROLES

Admin. Initializes the program, freezes and unfreezes globally, updates platform parameters, and proposes a new creator or resolver through a two-step handover.

Creator. Creates pools, registers streaks, sponsors gas as the only allowed external fee payer, claims for users after the self-claim window, and closes pool and prediction accounts.

Resolver. Finalizes pools by submitting the Merkle root, protocol fee, and winner count, and authorizes streak payouts.

Backend signer boundary. One in-scope backend service holds all four operational signers and drives every privileged flow, see L-05.

06 FINDINGS

What we found

ID	TITLE	SEVERITY	STATUS
H-01	Backend oracle can finalize pools with a stale BTCUSDT price from a capped aggregate-trades page	HIGH	FIXED
H-02	Streak reward claim transaction-binding bypass allows a double payout	HIGH	FIXED
M-01	Indexer consistency and resolution trust can lead to incorrect payout finalization	MEDIUM	FIXED
M-02	WASM reward cap waterfilling underpays uncapped winners	MEDIUM	FIXED
M-03	Non-idempotent streak reward settlement can duplicate payouts under failure	MEDIUM	FIXED
M-04	Non-qualifying streak participants are not reset until someone wins	MEDIUM	FIXED
M-05	Pool can be permanently bricked when a winner's leaf exceeds `max_roi`	MEDIUM	FIXED
M-06	Token-2022 ATA derivation mismatch can DoS `finalize_pool` on time-series pools	MEDIUM	FIXED
L-01	Access tokens remain valid for 41 days due to a millisecond expiry passed as JWT seconds	LOW	FIXED
L-02	Pyth volatility calculation accepts partial candle data and can distort precision scores	LOW	FIXED
L-03	Pool vault donations can block the shipped finalization helper before resolver submission	LOW	FIXED
L-04	Pool finalization can commit unreachable claim counts and deadlock cleanup	LOW	FIXED

06 FINDINGS

CONTINUED

ID	TITLE	SEVERITY	STATUS
L-05	Multiple privileged operational roles concentrated behind one backend signing boundary	LOW	ACKNOWLEDGED
L-06	`platform_fee` is dead config while `protocol_fee` is unbounded at finalization	LOW	FIXED
L-07	Unsafe account sizing via `std::mem::size_of` in init constraints can break upgrades	LOW	FIXED

HIGH SEVERITY · 2 FINDINGS

H-01 HIGH FIXED

Backend oracle can finalize pools with a stale BTCUSDT price from a capped aggregate-trades page

`BinanceService.getOutcome()` resolves a pool by fetching the last BTCUSDT aggregate trade before the pool reference timestamp. It queries a 120-second range and assumes the response holds the full range.

```
const startTime = Math.floor(outcomeMs - this.LOOKBACK_SECONDS * 1000);
const { data } = await firstValueFrom(
  this.httpService.get<BinanceAggTrade[]>(
    `${this.apiBaseUrl}/api/v3/aggTrades`,
    { params: { symbol: this.SYMBOL, startTime, endTime: outcomeMs, limit: 1000 } },
  ),
);
const before = data.filter((t) => t.T < outcomeMs);
if (before.length) {
  const last = before[before.length - 1];
  return Number(last.p);
}
```

The provider caps `/api/v3/aggTrades` at `limit = 1000`, and a `startTime + endTime` query returns the oldest records up to that cap. If the 120-second window holds more than 1000 trades, Trepa receives only the oldest page and treats its tail as the last trade before the reference time.

```
reference time = 12:00:00.000
query window   = 11:58:00.000 to 12:00:00.000, limit 1000
trades in range: 1800
provider returns: oldest 1000
Trepa selects: trade #1000 at 11:59:12
correct trade:  trade #1800 at 11:59:59.900
```

IMPACT

A stale outcome feeds pool resolution: winners, losers, and reward amounts are computed from it, stored, and committed on-chain through the resolver-signed `finalizePool`. The on-chain proof layer cannot tell a stale value from a correct one, so an entire pool settles on an economically wrong price. Live testing hit the 1000-row cap during normal BTCUSDT volume, and a capped response is still a successful HTTP response, so no retry or truncation warning fires.

RECOMMENDATION

Replace the wide historical query with a direct nearest-before lookup. The provider's `endTime` is inclusive, so query `endTime: outcomeMs - 1` with `limit: 1` and reject a trade whose drift exceeds a documented bound.

```
- startTime,
- endTime: outcomeMs,
- limit: 1000,
+ endTime: outcomeMs - 1,
+ limit: 1,
```

TEAM

Fixed in trepa-api PR #284. The query was switched to a direct nearest-before lookup using `endTime: outcomeMs - 1` and `limit: 1`, eliminating the page-truncation window so a stale boundary trade can no longer slip through.

H-02 HIGH FIXED

Streak reward claim transaction-binding bypass allows a double payout

The streak reward claim is a two-step flow. `POST /transactions/claim-streak-reward` builds an unsigned transaction for a specific reward, and the `/submit` endpoint accepts `streak_reward_id`, `signed_transaction`, and `proof`. On submit the backend verifies the signature and proof but never checks that the signed transaction corresponds to the `streak_reward_id` in the request. There is no binding between the signed transaction, the reward row, and the claim state.

1. Two unclaimed rewards exist for one streak: Reward A of `$50.15` and Reward B of `$25.05`.
2. The attacker creates a transaction for Reward A.
3. On submit, the attacker swaps `streak_reward_id` to Reward B while keeping the Reward A transaction and proof.
4. The backend accepts it. The DB marks Reward B claimed, but the chain executes the Reward A payout.
5. Reward A stays unclaimed in the DB, so the attacker claims it again for a second payout.

IMPACT

A repeatable double-claim that drains the streak pool. DB state and on-chain payout diverge, and each round leaves a still-claimable reward behind, so the primitive can be replayed against the streak accumulator balance.

TEAM

Fixed in trepa-api PR #268. The signed proof is now bound to a context object holding `streak_reward_id` and `wallet_address`, and the submit endpoint re-verifies that context before updating `is_claimed`, so swapping the reward id after signing no longer passes verification.

MEDIUM SEVERITY · 6 FINDINGS

M-01 MEDIUM FIXED

Indexer consistency and resolution trust can lead to incorrect payout finalization

Resolution trusts off-chain indexed data for prediction values, winner calculation, and reward distribution without verifying completeness. The indexer does not enforce event ordering, so an older backfill event can overwrite a newer live value. Live subscription and backfill run in parallel at startup, creating exactly that race. Resolution then checks only slot proximity, not whether every event was processed.

```
// libs/shared/src/services/web3.service.ts (validateIndexerSync)
abs(chain_slot - indexer_slot) < threshold
```

This checks proximity, not whether all events are processed, whether the data is complete, or whether any events were missed.

IMPACT

Incorrect indexed data produces an incorrect winner set and permanently incorrect on-chain payouts, with no completeness check to catch it before the resolver commits the Merkle root.

RECOMMENDATION

Give the indexer a deterministic ordering signal independent of slot and timestamp, and block resolution until the indexed state matches the chain.

TEAM

Fixed in trepa-onchain-svm PR #141 and trepa-api PR #291. A `prediction_revisions` counter was added to `PoolAccount`, incremented on every `update_stake` and `update_predict` and emitted in events, giving the indexer a deterministic ordering cursor. Resolution now blocks until the indexed revision matches the on-chain value, and the prediction-row count is validated against `open_predictions_left` before finalization.

M-02 MEDIUM FIXED

WASM reward cap waterfilling underpays uncapped winners

The WASM reward calculator can route winner-dividend funds into `takeout` while a winner is still below their gain cap. After choosing winners by accuracy it computes `dividends = loser_stakes - takeout` and splits them by weight, subject to each winner's cap. When one winner caps but another does not, the capped winner's excess should flow to the uncapped winner. Instead the residual is added to `takeout` and the uncapped winner is underpaid.

```
// trepa-api/libs/wasm/rust/src/lib.rs — get_final_alpha
let has_valid_cap_to_weight_ratio = cap_to_weight_ratios_values
    .iter()
    .any(|ratio| dividend_to_weight_ratio ⩽ *ratio);
let final_alpha = if has_valid_cap_to_weight_ratio {
    dividend_to_weight_ratio
} else if dividends ⩾ sum_gain_cap {
    // saturate at the largest cap
} else {
    // first valid alpha in theta-order
};
```

The `any()` uses raw proportional alpha if any single winner can take it without capping. Raw alpha is only safe if every winner can. With two winners staking `$1` each against `$190` of loser stake and one exact winner, the shortfall is concrete:

WINNER	CURRENT PAYOUT	CAP-AWARE PAYOUT
A (error 0)	$101	$101
B (error 99)	$3.41	$53
Takeout	**$87.59**	**$38**

About `$49.59` that should still go to an uncapped winner is instead added to takeout.

RECOMMENDATION

Replace the `any()` shortcut with `all()`, so raw alpha holds only when no winner caps. Otherwise the function falls through to the waterfilling path, which recomputes alpha for the remaining uncapped winners.

```
- let has_valid_cap_to_weight_ratio = cap_to_weight_ratios_values
-     .iter()
-     .any(|ratio| dividend_to_weight_ratio ≤ *ratio);
+ let raw_alpha_fits_every_winner_cap = cap_to_weight_ratios_values
+     .iter()
+     .all(|ratio| dividend_to_weight_ratio ≤ *ratio);
```

TEAM

Fixed in trepa-api PR #283. The waterfilling algorithm now redistributes capped winners' excess to still-uncapped winners instead of routing it to `takeout`, and explicitly fails on invalid or no-alpha inputs rather than silently falling back to zero.

M-03 MEDIUM FIXED

Non-idempotent streak reward settlement can duplicate payouts under failure

The streak claim flow is not idempotent. It marks the reward claimed in a DB transaction, sends the on-chain transaction, waits for confirmation, and rolls back the DB on confirmation failure. If the transaction confirms on-chain but the backend confirmation times out, the DB rollback leaves the reward unclaimed, so a retry pays a second time.

1. The user submits a claim.
2. The backend broadcasts the transaction to Solana.
3. The transaction succeeds on-chain.
4. The backend confirmation fails on an RPC timeout or network error.
5. The DB transaction rolls back and the reward stays unclaimed.
6. The user retries and a second payout occurs.

RECOMMENDATION

Make settlement idempotent. On a possibly-broadcast transaction, query chain status and mark the reward claimed if it confirmed, regardless of the API error, and add post-failure reconciliation.

TEAM

Fixed in trepa-onchain-svm PR #139 and trepa-api PR #289. The team went beyond the recommendation: an on-chain replay-protection PDA seeded with `(reward_id, streak_id, wallet)` now lets the chain itself reject duplicate claims, and the API passes `reward_id` to `claimStreakReward` while the indexer drives DB state from the emitted event.

M-04 MEDIUM FIXED

Non-qualifying streak participants are not reset until someone wins

`processStreakWinners` should increment qualifying users and reset failing users each round. In the mixed case where at least one user qualifies but nobody reaches `streak_count_required`, the function returns early before the reset block, so failing participants keep a stale streak count until a later round produces a winner.

```
current:  increment → check winners → return if none → reset
expected: increment → reset → check winners → return if none
```

1. `streak_count_required = 2`, and User A already has a count of 1 after qualifying in pool 1.
2. Pool 2: A fails the threshold, User B qualifies, but nobody wins. A should reset to 0, yet stays at 1 because the function returns early.
3. Pool 3: A qualifies again and reaches count 2, so A is wrongly treated as a streak winner though the real sequence was qualify, fail, qualify.

IMPACT

The backend can mint claimable `streak_rewards` rows for users who never had a consecutive streak, moving real value out of the streak pot to ineligible users and reducing `available_balance` against those invalid winners.

RECOMMENDATION

Move the non-qualifying reset above the no-winner early return, and restrict the winner query to wallets that qualified in the current pool.

TEAM

Fixed in trepa-api PR #272. The reset step was moved before the early-return check so non-qualifying participants reset every round, not only when someone wins, and the winner query was additionally restricted to wallets that qualified in the current pool.

M-05 MEDIUM FIXED

Pool can be permanently bricked when a winner's leaf exceeds `max_roi`

On-chain `claim_rewards` enforces a per-winner ROI cap and reverts with `MaxRoiExceeded` when `user_gains` exceed `stake * max_roi / 10000`.

```
if amount ⩾ ctx.accounts.prediction.stake {
    let user_gains = amount.checked_sub(ctx.accounts.prediction.stake)?;
    let gain_cap_amount = stake_u128.checked_mul(max_roi_u128)?.checked_div(10000)?;
    if user_gains > gain_cap_amount {
        return Err(CustomError::MaxRoiExceeded.into());
    }
}
```

The resolver side has no matching guard. `finalizePool.ts` only checks that the vault covers the total, and the WASM engine caps gains with a hardcoded `100x` instead of the pool's configured `max_roi`. Any `max_roi` below `100x` can produce a leaf that always reverts. Because the Merkle proof binds to the exact leaf, an over-cap winner can never claim, `claims_left` never reaches zero, and `close_pool_account` reverts.

1. Creator sets `max_roi = 100000` (10x). 1000 predictors stake 1 USDC each, so the vault holds 1000 USDC.
2. The price moves sharply and 5 winners emerge.
3. The resolver splits the pool into 200 USDC leaves.
4. The per-winner cap is `1 * 100000 / 10000 = 100` USDC of gain, so every claim reverts with `MaxRoiExceeded`.
5. `claims_left` stays at 5, `close_pool_account` reverts, and the 1000 USDC is permanently stranded. No attacker is required.

RECOMMENDATION

Enforce the per-winner cap at tree construction time, and bind the WASM distribution to the pool's real `max_roi`.

TEAM

Fixed. The WASM reward calculator now consumes the pool's configured `max_roi` instead of a hardcoded `100x` cap, so leaves can no longer be constructed above the on-chain `MaxRoiExceeded` boundary that would previously have stranded the vault.

M-06 MEDIUM FIXED

Token-2022 ATA derivation mismatch can DoS `finalize_pool` on time-series pools

`finalize_pool` for time-series pools validates the streak token account with `assert_valid_token_account`, which derives the expected ATA using legacy SPL `get_associated_token_address(authority, mint)`. For Token-2022 mints the correct address depends on the token program, so the derived address differs from the real Token-2022 ATA and finalize reverts with `InvalidAtaData` even when the caller passes the correct account.

```
[PoC] Finalize reverted with InvalidAtaData (code 6054): Invalid associated token account data
```

The resolver passes the correct Token-2022 streak ATA, yet the program still rejects it because its own derivation is program-agnostic. Finalization, and the downstream claims flow, is blocked for those pools.

RECOMMENDATION

Derive the expected ATA with a token-program-aware helper.

```
- let expected_ata = get_associated_token_address(authority.key, token_mint.key);
+ let expected_ata = get_associated_token_address_with_program_id(
+     authority.key, token_mint.key, token_program.key,
+ );
```

TEAM

Fixed in trepa-onchain-svm PR #115. The canonical derivation in `assert_valid_token_account` was switched to `get_associated_token_address_with_program_id`, so Token-2022 mints derive the correct address. Trepa does not currently use Token-2022 mints in production, so the fix lands defensively.

LOW SEVERITY · 7 FINDINGS

L-01 LOW FIXED

Access tokens remain valid for 41 days due to a millisecond expiry passed as JWT seconds

The API intends one-hour access sessions, but the JWT `exp` is roughly 1000x too long. `ACCESS_TOKEN_EXPIRY_MS = 60 * 60 * 1000` is correct for the Express cookie `maxAge`, which expects milliseconds, but the same value is also passed to JWT `expiresIn`, which `jsonwebtoken` reads as seconds.

```
get jwtSignOptions(): JwtSignOptions {
  return {
    secret: this.configService.getOrThrow<string>('JWT_SECRET'),
    expiresIn: ACCESS_TOKEN_EXPIRY_MS,
    issuer: 'trepa-api',
    audience: 'trepa-app',
    algorithm: 'HS256',
  };
}
```

`3,600,000` seconds is 41.67 days. The browser cookie drops after one hour, so the happy path looks correct, but a copied `trepa-token` value stays cryptographically valid for about 41.7 days because the JWT strategy enforces only the token `exp` and never reloads the user.

RECOMMENDATION

Use seconds for the JWT and keep milliseconds for the cookie, with distinct constants so the two units cannot be confused.

```
- expiresIn: ACCESS_TOKEN_EXPIRY_MS,
+ expiresIn: ACCESS_TOKEN_EXPIRY_SECONDS,
```

TEAM

Fixed in trepa-api PR #282. The JWT `expiresIn` value is now converted from milliseconds to seconds before being passed to the signer, restoring the intended one-hour access-token lifetime.

L-02 LOW FIXED

Pyth volatility calculation accepts partial candle data and can distort precision scores

`PythService.computeStdLogReturns()` computes the BTC 7-day 1-minute volatility (`std_log_returns`) that feeds `precision_score` , which drives streak qualification and reward-row creation. The only coverage check is `closes.length < 2` , so a handful of closes is accepted as a full 7-day sample of roughly 10,080 candles. It never verifies timestamp coverage, matching `t` and `c` lengths, or that the latest candle is near the requested end.

```
if (closes.length < 2) {
  throw new Error('Pyth 7d OHLC: insufficient data for std(log returns)');
}
```

IMPACT

A partial or stale volatility value flips streak qualification. The same 5% prediction error scores 184 at `std = 0.02` (does not qualify) and 508 at `std = 0.05` (qualifies and creates a reward). It cannot change the base pool winner set, so the impact is limited to the streak layer, hence Low.

RECOMMENDATION

Fail closed unless the response proves it covers the 7-day 1-minute window: reject non-ok chunks, require matching `t` and `c` lengths, require at least 95% coverage, and validate the boundary timestamps.

TEAM

Fixed in trepa-api PR #285. The volatility resolver now requires Pyth's response to cover at least 95% of the expected 7-day 1-minute window (about 10,080 samples) and validates the boundary timestamps before computing log returns, so under-covered or transient responses are rejected instead of distorting the precision score.

L-03 LOW FIXED

Pool vault donations can block the shipped finalization helper before resolver submission

The pool vault is an ordinary SPL token account, so anyone can transfer the stake mint into it. The on-chain program tolerates residual vault tokens and sweeps them at `close_pool_account`. The shipped finalization helper, however, treats the live vault balance as an exact settlement invariant: it compares the balance against `sum(prizes) + protocolFee` and throws when the vault holds more than expected by more than `ALLOWED_PRECISION_DEVIATION`. `resolvePool` runs this helper before submitting the resolver transaction, so a tiny donation blocks automatic resolution.

IMPACT

For a 6-decimal USDC pool, transferring slightly more than `10000` raw units, about one cent, makes the helper reject otherwise-valid settlement data before it reaches the on-chain `finalize_pool` instruction. No funds are stolen, but automatic resolution is griefed until an operator intervenes.

RECOMMENDATION

Compare against a stored `total_stake` rather than the live vault balance, and leave donations as residual for the close sweep.

TEAM

Fixed in trepa-onchain-svm PR #138, bundled with the L-06 cap-protocol-fee work. The settlement check now compares against a new on-chain `total_stake` field incremented inside `predict` and `update_stake`, not the live vault balance, so external donations remain as residual for `close_pool_account` to sweep and a one-cent donation can no longer brick finalization.

L-04 LOW FIXED

Pool finalization can commit unreachable claim counts and deadlock cleanup

`finalize_pool` stores the resolver-supplied `claims` scalar into `pool.claims_left` after only checking `claims > 0`. Nothing enforces that `claims_left` equals the number of uniquely reachable, still-unconsumed claims. Each successful `claim_rewards` decrements `claims_left` once and auto-closes its prediction account, and cleanup requires `claims_left == 0`. If finalization overstates the claim count, every reachable winner can finish and the pool can still remain permanently finalized but unclosable.

IMPACT

The affected pool sticks in a terminal state where cleanup, rent reclamation, and residual sweep never complete. Low, because it needs internally inconsistent resolver-submitted data rather than a permissionless trigger.

RECOMMENDATION

Reject any finalization where `claims > pool.open_predictions_left` before writing `claims_left`, and deduplicate winners in the resolver path.

TEAM

Fixed in trepa-onchain-svm PR #131 and trepa-api PR #287. `finalize_pool` now rejects any submission where `claims > open_predictions_left`, and the resolver path additionally deduplicates winners, rejects zero or negative prizes, and verifies that each winner's `PredictionAccount` exists and is unclaimed before committing the Merkle root.

L-05 LOW ACKNOWLEDGED

Multiple privileged operational roles concentrated behind one backend signing boundary

The on-chain program deliberately separates privileged actions. `FinalizePool` and `ClaimStreakRewards` require the resolver to sign, while post-window reward relays and pool close depend on the creator path. The in-scope backend `web3.service.ts` holds all four operational signers at once, `creatorKeypair`, `resolverKeypair`, `feeSponsorKeypair`, and `streakTopupKeypair`, and drives every privileged flow from one service. The on-chain checks work as designed, but one backend boundary concentrates domains the protocol otherwise separates.

IMPACT

Most exposed roles are operational or treasury-adjacent and can be contained by freeze or rotation once detected. The strongest lasting risk is resolver misuse before containment, because a malicious finalization or streak-claim authorization can land on-chain before the signer is rotated. Low, and not permissionless: exploitation requires compromise of the backend signing boundary before operators freeze or rotate.

RECOMMENDATION

Prioritize the resolver path. Move `resolvePool`, `constructClaimStreakRewardTransaction`, and `sendPresignedClaimStreakRewardTransaction` into a resolver-only worker injected with `resolverKeypair` and no other signer, then split the fee-sponsor, streak-topup, and creator relays into separate services. A code-level split only helps when backed by real runtime and secret separation, not just another class in the same process.

TEAM

Tracked as an ongoing infrastructure evaluation. The team is assessing remote signing services to separate resolver key custody from the rest of the backend, so this item stays open rather than being patched inside the remediation window.

L-06 LOW FIXED

`platform_fee` is dead config while `protocol_fee` is unbounded at finalization

`initialize` validates and stores `config.platform_fee`, but no runtime path enforces it. `finalize_pool` accepts a caller-supplied `protocol_fee` and transfers it from the pool vault to the treasury without checking it against `platform_fee`, the pool balance ratio, or any basis-point bound. The effective fee is therefore set entirely by resolver input, and `platform_fee` is misleading dead state.

IMPACT

Operators and users may assume `platform_fee` is the global cap, but on-chain it is unenforced. A malicious or buggy resolver can set an arbitrarily high `protocol_fee`, up to the available vault balance.

RECOMMENDATION

Either remove the `protocol_fee` argument and derive the fee from config, or keep it but strictly cap it by `pool.total_stake * pool.platform_fee / 10000` and document it as a requested fee capped by policy.

TEAM

Fixed across trepa-onchain-svm PRs #138, #140, #142 and trepa-api PRs #288, #293. The argument was kept for refund-style finalizations and capped on-chain at `pool.total_stake * pool.platform_fee / 10000` with a new `ProtocolFeeTooHigh` error. Follow-ups excluded max-ROI leftover from the settlement comparison and sourced the WASM fee rate from the pool's stored `platform_fee` instead of a hardcoded 20%.

L-07 LOW FIXED

Unsafe account sizing via `std::mem::size_of` in init constraints can break upgrades

The program allocates account space with `8 + std::mem::size_of::<T>()` instead of Anchor's serialized `8 + T::INIT_SPACE` across several `init` constraints (`ConfigAccount`, `PoolAccount`, `PredictionAccount`, `StreakAccumulatorAccount`). The structs are fixed-size today, so current impact is limited, but the codebase already carries upgrade-oriented `_reserved` fields. Any future variable-length field (`String`, `Vec`) would make `size_of` under-allocate and break initialization.

RECOMMENDATION

Derive `InitSpace` on the account structs and use `space = 8 + T::INIT_SPACE` in every `init`.

TEAM

Fixed in trepa-onchain-svm PR #130. All `init` constraints were switched from `8 + std::mem::size_of::<T>()` to Anchor's canonical `8 + T::INIT_SPACE` with `#[derive(InitSpace)]`, eliminating the future-upgrade footgun if variable-length fields are ever added.